Stress + Chaos — Phase 1.c: real `router.Build` /providers + /health (LVA-007)

Date: 2026-06-13. **Status:** RAN, verdict **PASS**, `go test` **EXIT=0**. **Builds on:** `DESIGN.md` (Phase 1 scaffold) + `EVIDENCE-phase1.md`.

What this phase adds

Phase 1 (`api_stress_test.go`) drove a **synthetic** Gin handler (a hand-rolled `/health`); `provider_middleware_stress_test.go` drove the real `ProviderMiddleware` in isolation. **Neither booted `internal/router.Build`** — the SINGLE production router that both `cmd/lava-api-go/main.go` and `internal/mobile` (the on-device embed) call. Phase 1.c closes that gap.

New file `lava-api-go/tests/stress/providers_router_stress_test.go` (`TestStressProvidersRouter`, build tag `stress`) boots `router.Build` via `httptest` and stresses the two **public, pre-auth** endpoints the Android onboarding wizard depends on — focused on `GET /providers`, the now-load-bearing catalogue endpoint (Crashlytics `47b000d5`, v1.3.4, was a real production failure where this fetch broke and the wizard fell back to bundled providers).

`Cfg=nil`/`AuthLadder=nil` is deliberate: with those nil, `router.Build` serves `/providers` + `/health` + `/ready` exactly as production does for an un-paired onboarding client (those routes are registered BEFORE the auth middleware). The Cache/Scraper route-resolution stubs (`router_stubs.go`, mirrored verbatim from the in-tree `internal/router/router_test.go`) are never touched by the catalogue path.

Dimensions

Dim	Name	N	Asserts (primary = user-visible)
SR1	router-providers-sustained	500	0×5xx; p99<250ms; **parsed JSON body provider-count == 2** (50-sample)

Dim	Name	N	Asserts (primary = user-visible)
SR2	router-health-sustained	300	0×5xx; p99<250ms; liveness 200
SR3	router-providers-concurrent	64	all complete; all 200; **all bodies right count** (no torn registry read); `-race` clean
CR1	dependency-down isolation+recovery	500	chaos — see below

CR1 chaos injects a real fault at a **real seam**: `chaosReadiness.probe` is the exact `observability.ReadinessProbe` `router.Build` wires into `GET /ready`. Flipping it down mid-stream simulates the real DB/breaker going unavailable. The test asserts graceful degradation, NOT a cascade:

- `/ready` returns a clean **503** for all 150 fault-window requests (no panic/hang);
- `/health` + `/providers` **stay 200 with correct bodies** throughout (fault isolated);
- `/ready` **recovers to 200 within 1 request** after the fault clears.

The `firebase noop RecordNonFatal ... 503 on GET /ready` lines in the run output are REAL production §6.AC telemetry firing — proof the chaos request crossed the actual production middleware chain, not a synthetic handler.

SLO + measured numbers (verbatim from the run)

SLO: `0 × 5xx` under load + `p99 < 250ms` + correct catalogue body.

```
SR1: 500 req ~6.6k req/s  p50 0.137ms p95 0.219ms p99 0.322ms max
1.012ms  errRate 0.000  5xx 0
SR2: 300 req              p50 0.109ms p95 0.184ms p99 0.249ms max
0.305ms  errRate 0.000  5xx 0
SR3: 64 concurrent        p50 0.960ms p95 1.606ms p99
1.770ms              all-complete  5xx 0  -race clean
CR1: /ready 503=150/150 during fault | /health+/providers bad=0
5xx=0 | recovery=1 req
```

Leak guards: goroutines 3→4, open FDs 0→0. Verdict **PASS**.

Falsifiability (all reverted — see `.lava-ci-evidence/chaos-stress/2026-06-13-providers-stress-run.txt`)

- **Mutation A** — `time.Sleep(300ms)` in `ProvidersHandler.GetProviders` → SR1 FAILED `p99=305.83ms(SLO<250)`. The latency SLO trips on a real hot-path bottleneck.
- **Mutation B** — `chaosReadiness.probe` never recovers → CR1 FAILED `recov=0 (want 1)`, while SR1/SR2/SR3 stayed PASS (failure **localized** to the recovery path).
- **Mutation C** — `GetProviders` returns an empty list → SR1 `badBody=50`, SR3 `concBad=64`, CR1 `healthyBad=150` — all status 200 but every parsed body wrong. This is the exact empty-but-200 shape of the Crashlytics-`47b000d5` fallback class; the body-content assertion (not status) is what catches it (§6.AB).

Run command

```
cd lava-api-go
GOMAXPROCS=2 nice -n 19 go test -tags stress -count=1 -v \
    -run TestStressProvidersRouter ./tests/stress/...
# also: ... -race ...   (no data race in the concurrent registry read)
```

Build prerequisite on a fresh worktree

`lava-api-go/go.mod` replaces `digital.vasic.*` to `../submodules/*`. A git worktree created without initialized submodules has an empty `submodules/`. Symlink the required submodule trees read-only from the parent checkout before running (the §11.4.79 own-org-submodule pattern prior agents used):

```
for d in auth cache challenges concurrency config containers
         database discovery \
          helixqa http3 mdns middleware observability ratelimiter
         recovery security; do
  ln -s "/abs/path/to/parent/submodules/$d" "submodules/$d"
done
```

Honest status

- §11.4.85 in-process minimums (N≥100 sustained with p50/p95/p99; N≥10 concurrent all-complete no-race; fault injection + recovery on

user-visible state): **MET** by this phase against the REAL production router.

- C3 (Postgres-kill) + C4b (pool-exhaustion) remain **OPERATOR_GATED** in the sibling `api_stress_test.go` (need real Postgres under podman); recorded `ran=false`, NOT faked.
- Constitution pin NOT bumped (CONST-049 / operator-gated). Version files NOT touched (1.3.7 queue). `–tags realtrackers` NOT run (no outbound calls).